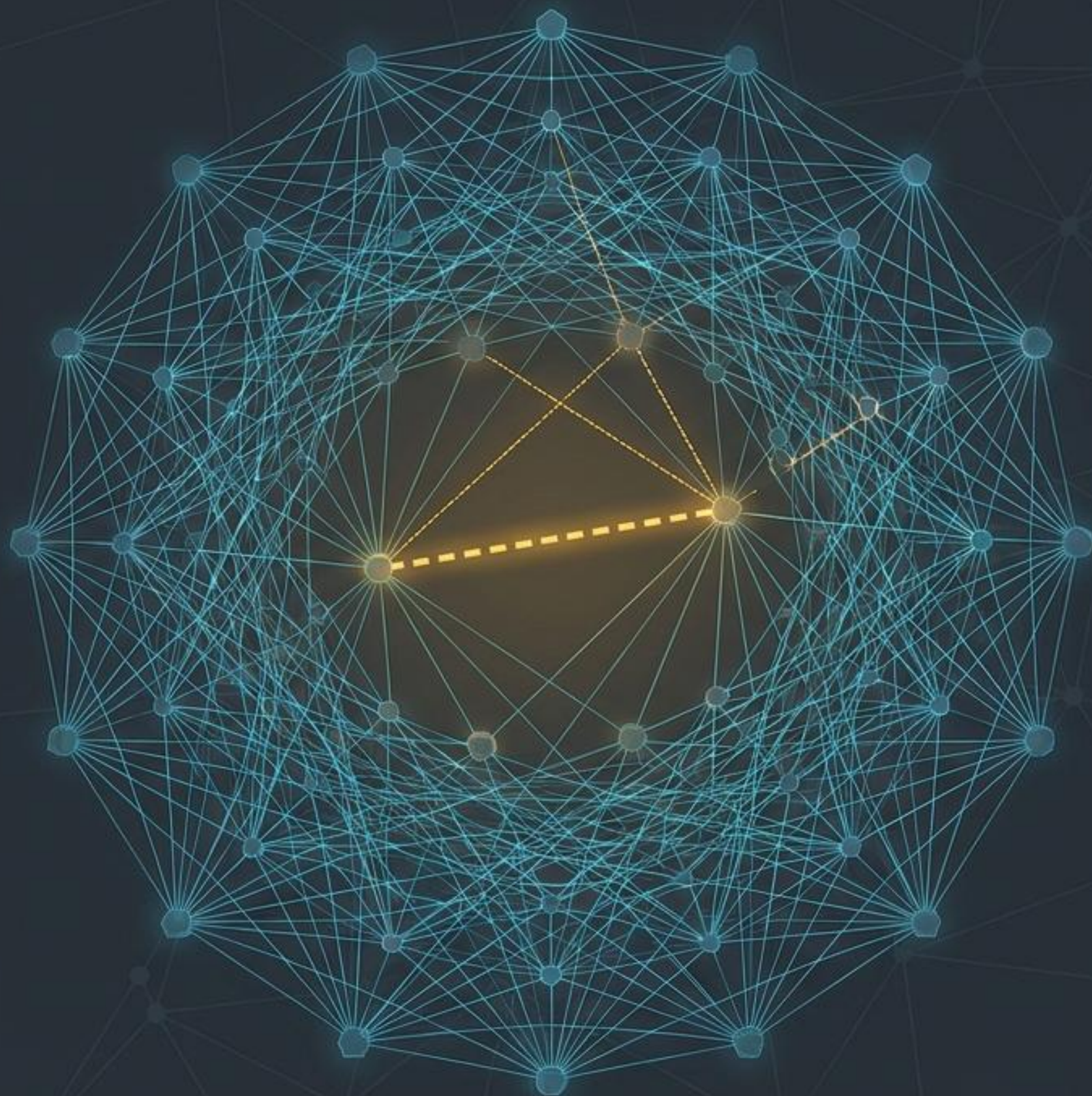


Grafowe Uczenie Maszynowe

Projektowanie architektury predykcyjnej dla grafów wiedzy (Link Prediction) w praktyce.

dr Zbigniew Galar

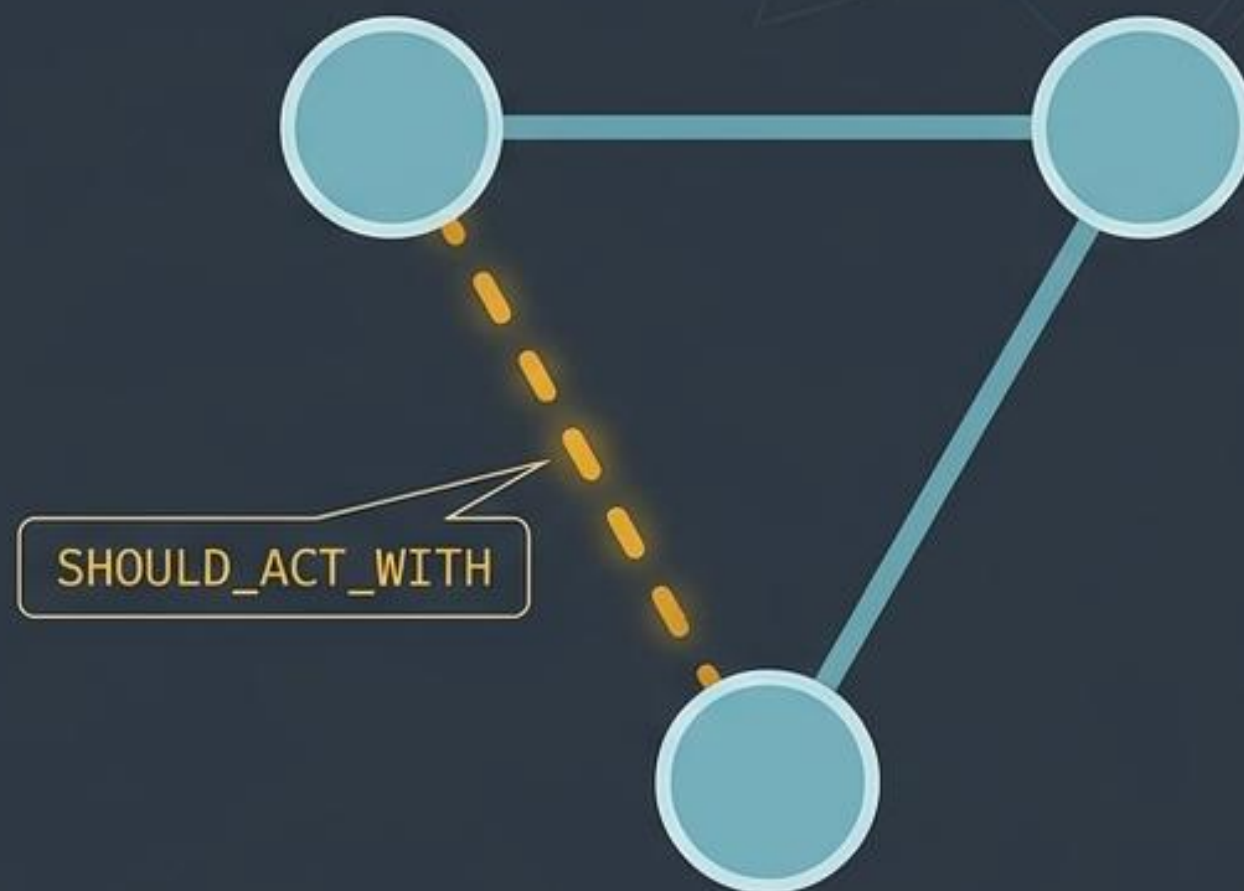


Topologia jako źródło prawdy

Tradycyjne modele ML widzą dane jako płaskie tabele. Grafowe Uczenie Maszynowe (GML) wykorzystuje samą strukturę relacji do generowania wiedzy.

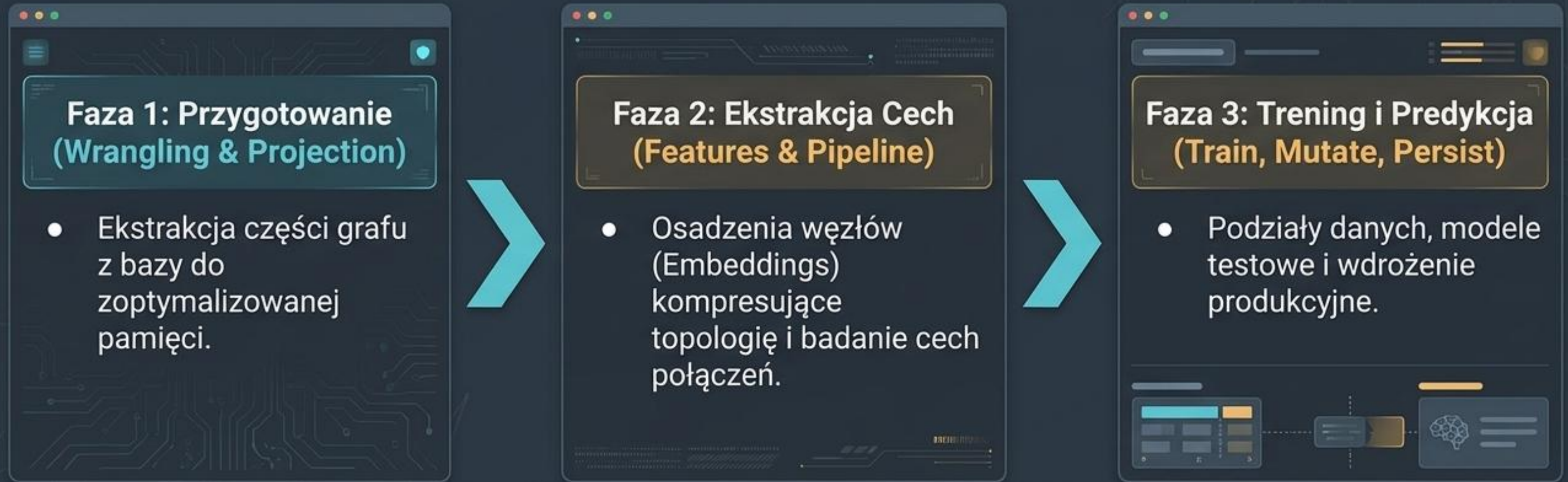


Wyzwanie: Kogo powinniśmy obsadzić w nowym filmie, bazując na ukrytych wzorcach historycznych współprac aktorskich?



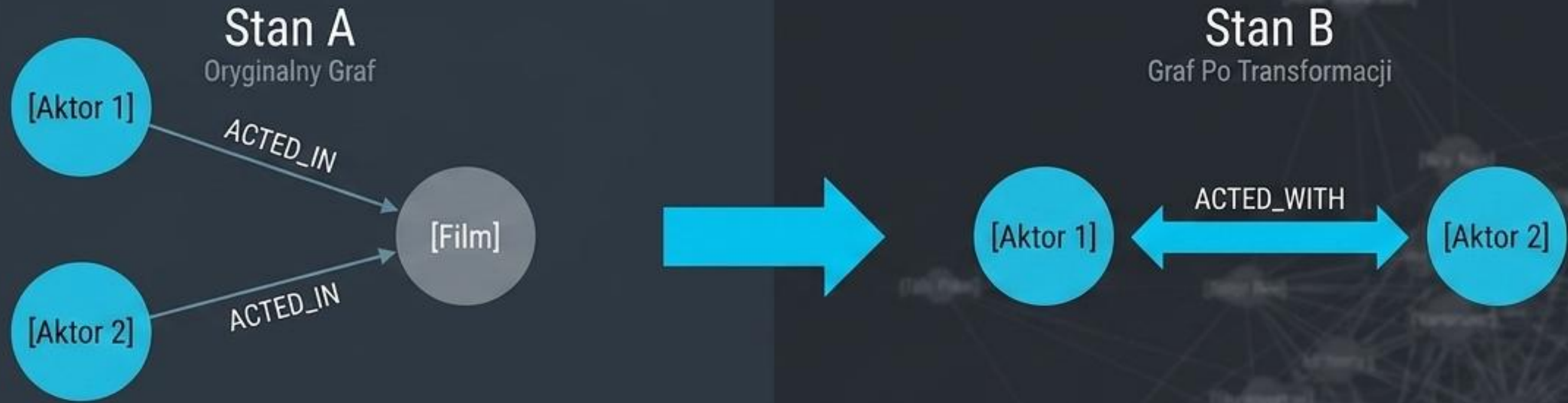
Zamiast ręcznie analizować miliony ścieżek, zbudujemy predykcyjny rurociąg uczenia maszynowego (*Link Prediction*).

Deklaratywna anatomia rurociągu ML



Uwaga: Ponieważ ta funkcjonalność jest wbudowana w Neo4j, skupiamy się wyłącznie na deklaracji konfiguracji rurociągu, omijając ciężką implementację algorytmiczną.

Faza 1: Porządkowanie danych przed treningiem



```
MATCH (a:Person)-[:ACTED_IN]->(Movie)<-[:ACTED_IN]-(b:Person)
MERGE (a)-[:ACTED_WITH]->(b)
```

Tworzymy pojedynczą, bezpośrednią relację historyczną – jest to jedyna operacja transformacji (data wrangling) przed uruchomieniem ML.

Faza 1: Rzutowanie do przestrzeni analitycznej

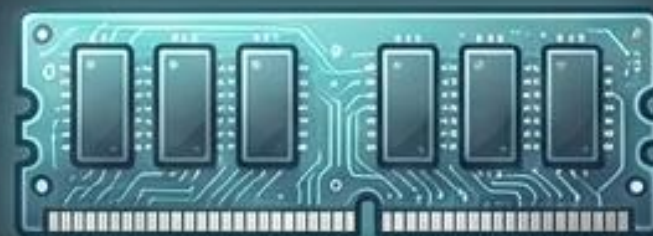
Knowledge Graph



(Dysk Twardy)



actors-graph

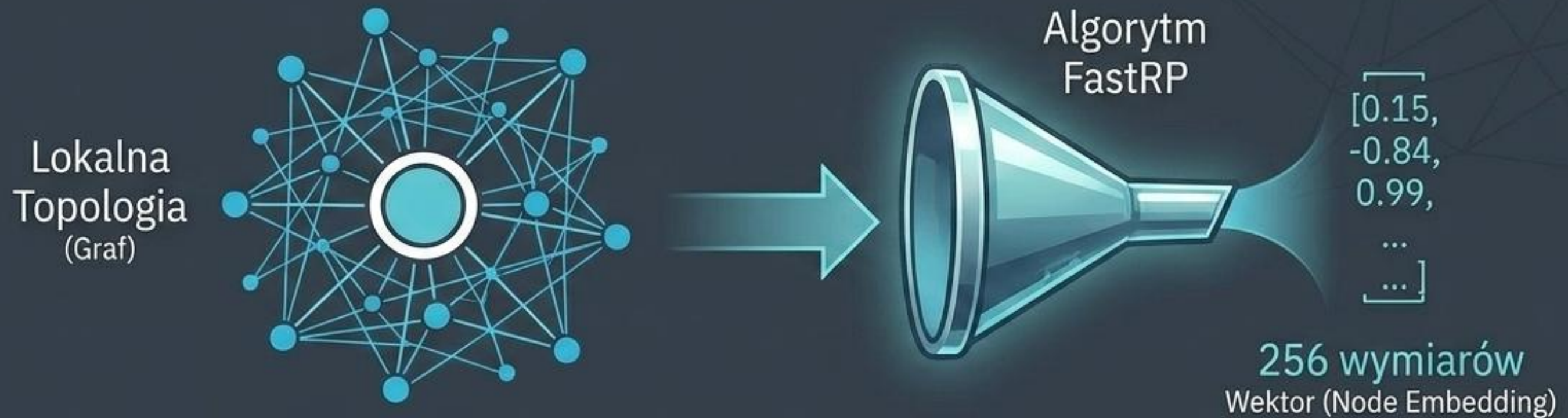


Tworzymy rzut węzłów w pamięci podręcznej.

```
CALL gds.graph.project(  
  'actors-graph', { Person: {} },  
  { ACTED_WITH: { orientation: 'UNDIRECTED' } }  
)  
  
CALL gds.beta.pipeline.linkPrediction.create('actors-pipeline')
```

Kierunek relacji
'kto z kim grał' nie
ma tu znaczenia
– optymalizacja
obliczeń.

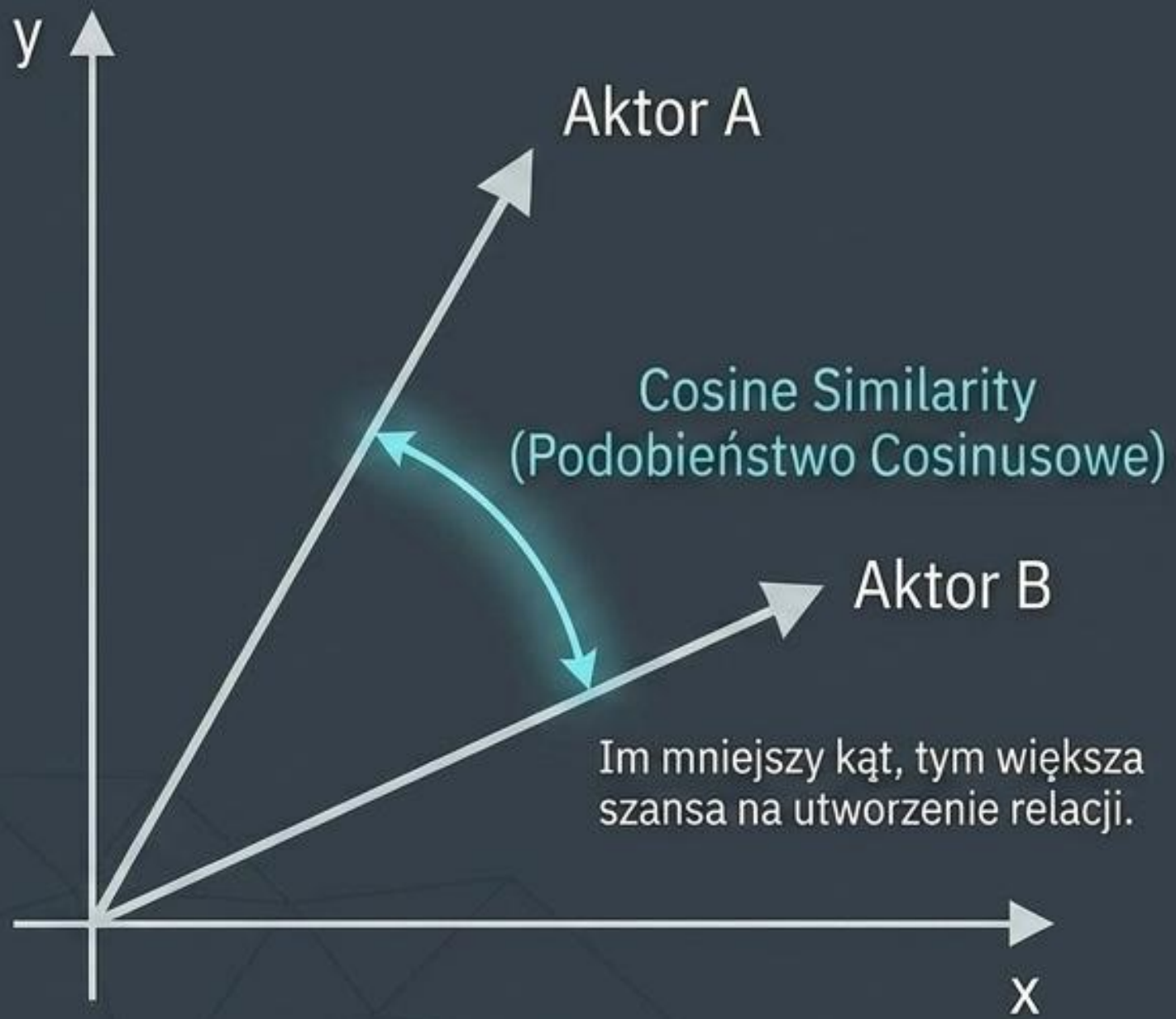
Faza 2: Kodowanie topologii w wektorach (Node Embeddings)



```
CALL gds.beta.pipeline.linkPrediction.addNodeProperty(  
  'actors-pipeline', 'fastRP', {  
    mutateProperty: 'embedding',  
    embeddingDimension: 256,  
    randomSeed: 42  
  }  
) }
```

Algorytm kompresuje unikalną, lokalną architekturę węzła do 256-wymiarowej reprezentacji liczbowej, gotowej dla modeli ML.

Faza 2: Obliczanie prawdopodobieństwa połączeń



```
CALL gds.beta.pipeline.linkPrediction.addFeature(  
    'actors-pipeline', 'cosine', {  
        nodeProperties: ['embedding']  
    }  
) YIELD featureSteps
```

Hadamard Product

L2

Zamiast Cosine, rurociąg obsługuje również inne modele matematyczne.

Faza 3: Segmentacja danych i Auto-Tuning



```
CALL gds.beta.pipeline.linkPrediction.configureSplit(  
  'actors-pipeline', { testFraction: 0.25, trainFraction: 0.6, validationFolds: 3 } )  
  
CALL gds.alpha.pipeline.linkPrediction.configureAutoTuning(  
  'actors-pipeline', {maxTrials: 100})
```

System wykona do 100 iteracji, aby automatycznie odnaleźć optymalne parametry dla wybranego modelu.

[Logistical Regression]



[Logistical
Regression]



[Random
Forest]



[Multilayer
Perceptron]

Faza 3: Walidacja zasobów i proces treningowy

Uruchomienie Treningu

Krok Bezpieczeństwa

```
CALL gds.beta.pipeline.linkPrediction.train.estimate(...)  
YIELD requiredMemory
```



Safety Check: OK ✓

```
CALL gds.beta.pipeline.linkPrediction.train(  
  'actors-graph', {  
    pipeline: 'actors-pipeline',  
    modelName: 'actors-model',  
    metrics: ['AUCPR'],  
    targetRelationshipType: 'ACTED_WITH'  
  })  
YIELD modelInfo
```

avgTrainScore

0.92

outerTrainScore

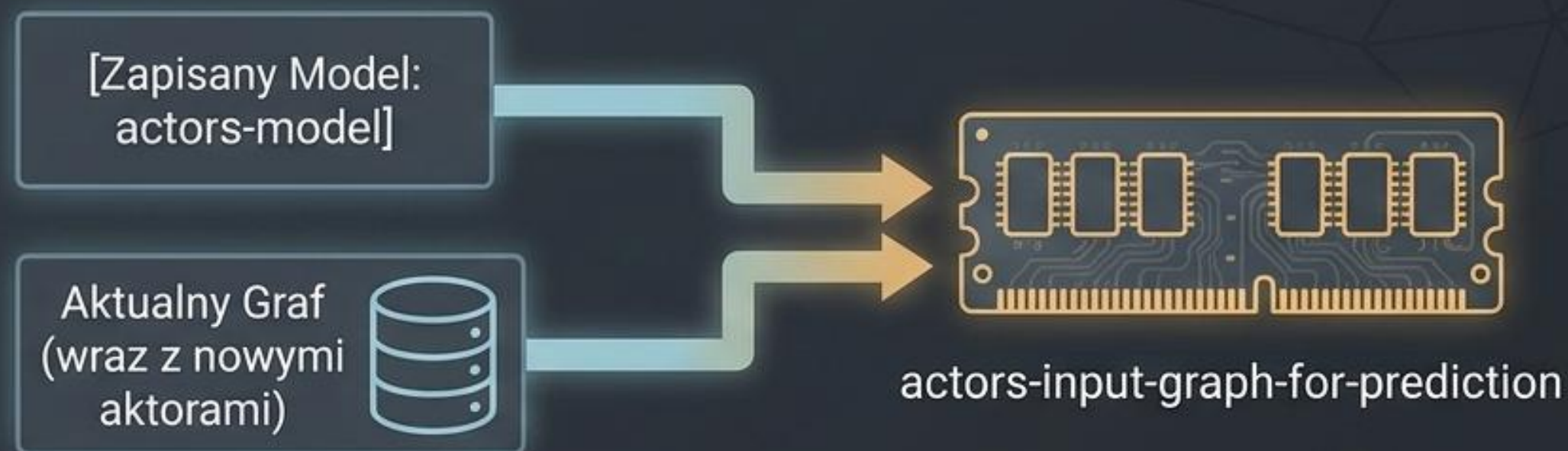
0.89

testScore

0.91

Narzędzia oceny jakości wytrenowanego modelu AUCPR.

Faza 4: Przejście do środowiska produkcyjnego



```
CALL gds.graph.project(  
  'actors-input-graph-for-prediction', { Person: {} },  
  { ACTED_WITH: { orientation: 'UNDIRECTED' } }  
)
```

Zanim zaaplikujemy model, rzutujemy nowy graf operacyjny zawierający świeże, niezbadane dane ("perturbacje").

Faza 4: Egzekucja modelu i zapis predykcji

```
CALL gds.beta.pipeline.linkPrediction.predict.mutate(  
  'actors-input-graph-for-prediction', {  
    modelName: 'actors-model',  
    relationshipTypes: ['ACTED_WITH'],  
    mutateRelationshipType: 'SHOULD_ACT_WITH',  
    topN: 20,  
    threshold: 0.4  
  })
```

Docelowy typ naszej
nowej relacji
rekomendacyjnej.

Twardy limit
wyników per węzeł.

Próg zaufania dla predykcji.
Celowo niski w modelu edukacyjnym
– na produkcji wymaga
podbicia w celu redukcji błędów.

Cykl Życia Wyników: Strumień czy Zapis?

Ścieżka A: Zapis do Bazy (Persist)



Ścieżka B: Aplikacja Front-endowa



○○○

```
CALL gds.beta.graph.relationships.stream(...)
MERGE(source)-[:SHOULD_ACT_WITH]->(target)
```

Odpalamy zapytanie bez zapisu do grafu.
Wyniki idą od razu na ekran użytkownika.

Normalizacja Grafu

○○○

```
1 WHERE id(a) > id(b)
2 DELETE d
```

Usuwamy **zduplikowane, lustrzane krawędzie** (graf staje się zoptymalizowany pod kątem relacji symetrycznych).

Efekt końcowy: Odkrycie ukrytej wiedzy

```
MATCH (a:Person)-[:SHOULD_ACT_WITH]->(b:Person)
RETURN a.name, b.name
```



Rekomendacje Obsady

Aktor X ↔ Aktor Y

Aktor Z ↔ Aktor W

Aktor P ↔ Aktor Q

Aktor R ↔ Aktor S

Aktor M ↔ Aktor N

Cała złożoność wektoryzacji i topologicznego ML zamknęła się w powrocie do najprostszego zapytania w języku Cypher. Graf powiększył się o **predykcyjną inteligencję**.

Architektura ciągłego wnioskowania

Macierz Stanów Grafu

Stan 0: Baza Surowa



Płaskie, historyczne fakty
na dysku twardym.

Stan 1: Rzutowanie RAM i Modele



Przestrzeń obliczeniowa,
ewaluacja wektorowa,
auto-tuning.

Stan 2: **Inteligentny Graf**



Baza wzbogacona o silnik
rekomendacyjny
(Real-time querying).

Neo4j ukrywa złożoność operacyjną. Twoim jedynym zadaniem jest **zdefiniowanie architektury wiedzy i określenie celów predykcyjnych**. System **robi resztę**.